

Feature/Training/Inference (FTI) architecture

The pattern suggests that any AI/ML system can be boiled down to these three pipelines: feature, training, and inference.

The **feature pipelines** take raw data as input and output features and labels to train our model(s).

The **training pipeline** takes the features and labels from the feature store as input and outputs our trained model(s).

The **inference pipeline** inputs the features and labels from the feature store and the trained model(s) from the model registry. Using these two components, we can make predictions in batch or real-time mode and serve them to the client.

The feature pipeline

The feature pipelines take as input data and output features & labels used to train the model.

Instead of directly passing them to the model, the features and labels are stored inside a feature store. Its responsibility is to store, version, track, and share the features.

By saving the features into a feature store, we always have a state of our features. Thus, we can easily send the features to the training and inference pipeline(s).

As the data is versioned, we can always ensure that the training and inference time features match. Thus, we avoid the training-serving skew problem.

The training pipeline

The training pipeline takes the features and labels from the features store as input and outputs a train model or models.

The models are stored in a model registry. Its role is similar to that of feature stores, but this time, the model is the first-class citizen. Thus, the model registry will store, version, track, and share the model with the inference pipeline.

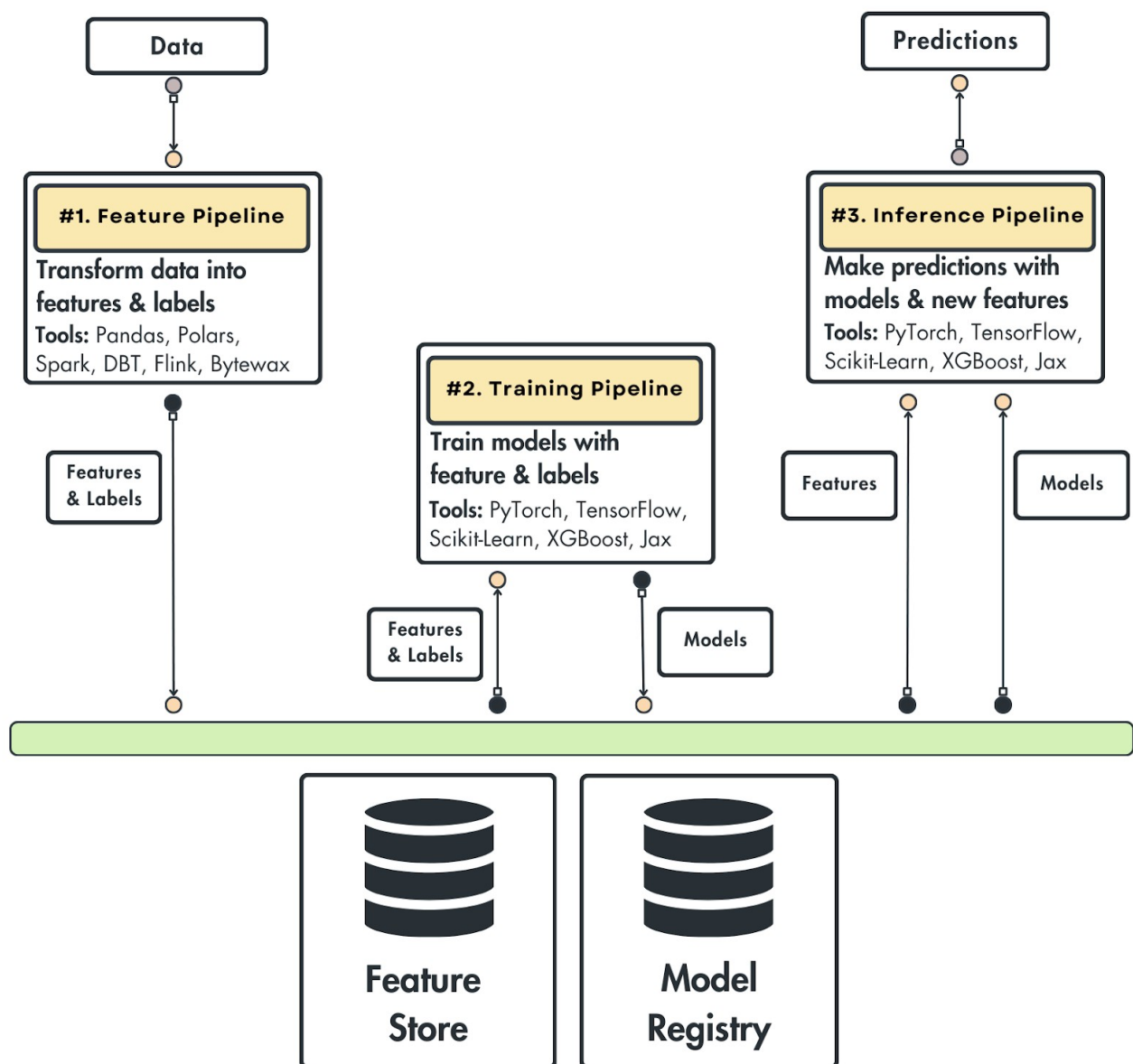
Also, most modern model registries support a metadata store that allows you to specify essential aspects of how the model was trained. The most important are the features, labels and their version used to train the model. Thus, we will always know what data the model was trained on.

The inference pipeline

The inference pipeline takes as input the features & labels from the feature store and the trained model from the model registry. With these two, predictions can be easily made in either batch or real-time mode.

As this is a versatile pattern, it is up to you to decide what you do with your predictions. If it's a batch system, they will probably be stored in a database. If it's a real-time system, the predictions will be served to the client who requested them.

As the features, labels, and model are versioned. We can easily upgrade or roll back the deployment of the model. For example, we will always know that model v1 uses features F1, F2, and F3, and model v2 uses F2, F3, and F4. Thus, we can quickly change the connections between the model and features.



To conclude, the most important thing you must remember about the FTI pipelines is their interface:

- The feature pipeline takes in data and outputs features & labels saved to the feature store.

- The training pipelines query the features store for features & labels and output a model to the model registry.
- The inference pipeline uses the features from the feature store and the model from the model registry to make predictions.

It doesn't matter how complex your ML system gets. These interfaces will remain the same.

Production pipeline components

we have 5 significant components that we have to understand:

1. The data pipelines
2. The feature pipelines
3. The training pipeline
4. The inference pipelines
5. The observability pipeline

You might wonder why we have five pipelines instead of three, as the FTI architecture suggests.

The data engineering team often owns the data pipeline, which prepares the data required to build AI systems.

Meanwhile, the observability pipeline is implemented on top of the FTI architecture to monitor and evaluate the system. Having eyes and ears all over your system is critical for success, especially in LLM systems, which are highly non-deterministic.

The data pipelines

As the name suggests, the data pipelines collect data from Notion, clean it, standardize it, and load it to a document database as our clean and normalized data snapshot before feeding it into the feature pipelines.

By storing the standardized and cleaned data in a document database, such as **MongoDB**, we collect the data once (as a backup), which we can use to experiment and build our AI system.

This is usually done with a data warehouse or lake in larger AI systems, but a document database is lighter and does the job for us.

The data pipeline is split into two significant components.

1. **The data collection pipeline:** It uses Notion's API to retrieve the data programmatically, where each Notion data entry is standardized to Markdown format and saved as JSON along with necessary metadata. To decouple the data collection step from the rest of the system, we save everything to a public S3 bucket to avoid giving public access to our Notion workspace.
2. **The ETL pipeline:** It extracts the raw Notion documents from S3, finds all the embedded links within the documents, crawls them, and standardizes them into Markdown format. It also computes a quality score/document using LLMs. Ultimately, it saves all the documents and metadata into MongoDB.

The feature pipelines

The feature pipelines leverage the standardized and clean data from [MongoDB](#) for two things:

- To populate a [MongoDB vector index](#) for doing RAG.
- To generate a summarization instruct dataset for fine-tuning an LLM.

These are two standard feature pipelines you will see in the GenAI world.

Within the RAG feature pipeline, we will implement advanced pre-retrieval RAG techniques, such as Contextual Retrieval, proposed by Anthropic. To implement it, we will require a summarization LLM and hybrid search.

Ultimately, we chunk the documents, embed them, and load them into the [MongoDB vector index](#).

Using APIs such as OpenAI for summarization can get costly (which we need for Contextual Retrieval), so we fine-tune a summarization open-source LLM. To do this, we require a custom summarization dataset.

Thus, we will leverage distillation techniques for the dataset generation pipeline to create a high-quality summarization instruction dataset based on our documents.

We will save the generated dataset to Hugging Face's data registry.

The training pipelines

The training pipeline reads the instruct dataset from the data registry and uses **Unsloth** to fine-tune a Llama 3.1 8B LLM. We use **Comet** to log the metrics and hyperparameters between multiple experiments, compare them, and pick the best one.

After deciding on the best model, we load it into Hugging Face's model registry. As an example, you can check our fine-tuned

LLM: **Pauliusztin/Meta-Llama-3.1-8B-Instruct-Second-Brain-Summarization**

ZenML orchestrates and manages the data, feature, and training pipelines, helping us run the pipelines with a clearly defined structure and configuration. As illustrated in Figure 9, we can track the progress, status and history of each pipeline in a beautiful UI.

The inference pipelines

The summarization inference pipeline, which contains only the fine-tuned LLM, is deployed as a real-time inference endpoint on **Hugging Face's Dedicated Endpoints** serverless service.

The agentic inference pipeline is our AI assistant, which takes as input requests from a user and provides answers leveraging the data from the **MongoDB vector database**.

We implemented it using Hugging Face's **smolagents** Python framework, which allows us to build agents without hiding too much of what is going on behind the scenes. We attached a retriever tool that interacts with the vector database and a summarization tool to help us synthesize answers.

The observability pipelines

The last piece of the puzzle is the observability pipeline, which consists of two main components:

- Prompt monitoring
- LLM evaluation

For both, we will use **Opik**, which provides a beautiful dashboard for monitoring complex prompt traces, as seen in Figure 11.

It also provides a Python SDK to help us evaluate agentic and RAG applications, track the results and compare them (similar to experiment tracking, but for evaluating LLM applications).